

Отчет по лабораторной работе №1

**Полнодоступная двухсервисная модель Эрланга с одинаковыми
интенсивностями обслуживания**

Иванов Сергей Владимирович, НПИбд-01-23

Содержание

1	Теоретические сведения	5
2	Алгоритм расчета	7
3	Численный анализ	8
3.1	Импорт библиотек	8
3.2	Исходные данные	9
3.3	Функция расчета характеристик модели	9
3.4	Расчет характеристик для заданных значений	11
3.5	Зависимость вероятности блокировки от интенсивности	11
3.6	Зависимость среднего числа запросов от интенсивности	13
4	Вывод	15

Список иллюстраций

3.1	Импорты	8
3.2	Исходные значения C , λ_1 , λ_2 и μ	9
3.3	Функция <code>erlang_model</code>	10
3.4	Вызов функции и полученные значения p , E , B_1 , B_2 и N	11
3.5	Цикл расчета E , B_1 , B_2 и график вероятностей блокировки	12
3.6	График вероятностей блокировки	13
3.7	Цикл расчета N	14
3.8	График среднего числа запросов	14

Список таблиц

1 Теоретические сведения

Исследуется полнодоступная двухсервисная модель Эрланга. Система имеет емкость C и обслуживает запросы двух типов, поступающие с интенсивностями λ_1 и λ_2 .

Для обоих типов запросов интенсивность обслуживания одинакова:

$$\mu_1 = \mu_2 = \mu.$$

Интенсивности предложенной нагрузки:

$$\rho_1 = \frac{\lambda_1}{\mu}, \quad \rho_2 = \frac{\lambda_2}{\mu}.$$

Суммарная нагрузка:

$$\rho = \rho_1 + \rho_2.$$

Состояние системы определяется числом одновременно обслуживаемых запросов:

$$X = \{0, 1, \dots, C\}.$$

Если система находится в состоянии C , все ресурсы заняты, поэтому новый запрос блокируется.

Стационарное распределение вероятностей:

$$p_n = \frac{\rho^n/n!}{\sum_{i=0}^C \rho^i/i!}, \quad n = 0, 1, \dots, C.$$

Вероятность блокировки по времени:

$$E_1 = E_2 = E = p_C.$$

Вероятность блокировки по вызовам для каждого типа:

$$B_i = \frac{\lambda_i}{\lambda_1 + \lambda_2} E.$$

Вероятность блокировки по нагрузке:

$$C_1 = C_2 = E.$$

Среднее число обслуживаемых запросов:

$$N = \sum_{n=0}^C n p_n.$$

2 Алгоритм расчета

1. Задать C, λ_1, λ_2 и μ .
2. Рассчитать ρ_1, ρ_2 и суммарную нагрузку ρ .
3. Для каждого состояния системы рассчитать вес $\rho^n/n!$.
4. Нормировать полученные значения и получить вероятности p_n .
5. Определить вероятность блокировки по времени $E = p_C$.
6. Рассчитать вероятности блокировки по вызовам B_1 и B_2 .
7. Рассчитать среднее число запросов N .
8. Изменяя λ_1 , построить зависимости вероятностей блокировки и среднего числа запросов от интенсивности входящего потока.

3 Численный анализ

В программе используются следующие исходные значения:

$$C = 10, \quad \lambda_1 = 4, \quad \lambda_2 = 3, \quad \mu = 1.$$

Именно эти параметры используются в программе.

3.1 Импорт библиотек

Для вычислений используется библиотека NumPy, а для построения графиков — Matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt
✓ [1] 816ms
```

Рисунок 3.1: Импорты

3.2 Исходные данные

```
C = 10
lambda1 = 4.0
lambda2 = 3.0
mu = 1.0
✓ [2] < 10 ms
```

Рисунок 3.2: Исходные значения C, lambda1, lambda2 и mu

3.3 Функция расчета характеристик модели

В программе все основные характеристики рассчитываются одной функцией `erlang_model`.

Сначала проверяется корректность исходных данных. Затем рассчитываются нагрузки ρ_1 , ρ_2 и их сумма ρ .

Для вычисления стационарных вероятностей сначала формируется массив весов. Первый вес равен 1, а каждый следующий рассчитывается через предыдущий:

$$w_n = w_{n-1} \frac{\rho}{n}.$$

Такой способ соответствует выражению $\rho^n/n!$, но позволяет не вычислять степень и факториал отдельно для каждого состояния.

После этого веса нормируются:

$$p_n = \frac{w_n}{\sum w_n}.$$

Последний элемент массива вероятностей является вероятностью блокировки:

$$E = p_C.$$

Затем рассчитываются B_1 , B_2 и среднее число запросов N .

```
def erlang_model(C, lambda1, lambda2, mu):

    rho1 = lambda1 / mu # нагрузка потока
    rho2 = lambda2 / mu
    rho = rho1 + rho2

    weights = [1.0]
    for n in range(1, C + 1):
        weights.append(weights[-1] * rho / n)

    weights = np.array(weights)
    p = weights / weights.sum()

    E = p[-1]

    if lambda1 + lambda2 == 0:
        B1 = 0.0
        B2 = 0.0
    else:
        B1 = lambda1 / (lambda1 + lambda2) * E
        B2 = lambda2 / (lambda1 + lambda2) * E

    N = sum(n * p[n] for n in range(C + 1))

    return p, E, B1, B2, N

✓ [3] < 10 ms
```

Рисунок 3.3: Функция erlang_model

3.4 Расчет характеристик для заданных значений

Функция возвращает пять значений:

- p — массив стационарных вероятностей;
- E — вероятность блокировки по времени;
- $B1$ — вероятность блокировки по вызовам для первого типа;
- $B2$ — вероятность блокировки по вызовам для второго типа;
- N — среднее число запросов в системе.

```
p, E, B1, B2, N = erlang_model(C, lambda1, lambda2, mu)

print("Распределение вероятностей:")
for n, pn in enumerate(p):
    print(f"p[{n}] = {pn:.6f}")

print()
print(f"E1 = E2 = {E:.6f}")
print(f"B1 = {B1:.6f}")
print(f"B2 = {B2:.6f}")
print(f"C1 = C2 = {E:.6f}")
print(f"Среднее число запросов N = {N:.6f}")
print(f"Сумма вероятностей = {p.sum():.6f}")
✓ [4] < 10 ms
```

Рисунок 3.4: Вызов функции и полученные значения p , E , $B1$, $B2$ и N

Дополнительно выводится сумма всех стационарных вероятностей. Она должна быть равна 1, что позволяет проверить правильность нормировки распределения.

3.5 Зависимость вероятности блокировки от интенсивности

Для исследования влияния интенсивности входящего потока изменяется λ_1 от 0 до 20. Значения C , λ_2 и μ остаются постоянными.

В строке

```
_, e, b1, b2, _ = erlang_model(C, l1, lambda2, mu)
```

функция снова возвращает пять значений. Для этого графика нужны только E, B1 и B2. Поэтому первое значение ρ и последнее значение N записываются в `_`, то есть фактически не используются.

Полученные значения сохраняются в трех списках:

- E_values;
- B1_values;
- B2_values.

После этого все три зависимости строятся на одном графике.

```
lambda1_values = np.linspace(0, 20, 200)

E_values = []
B1_values = []
B2_values = []

for l1 in lambda1_values:
    _, e, b1, b2, _ = erlang_model(C, l1, lambda2, mu)
    E_values.append(e)
    B1_values.append(b1)
    B2_values.append(b2)

plt.figure(figsize=(9, 5))
plt.plot(lambda1_values, E_values, label="E1 = E2")
plt.plot(lambda1_values, B1_values, label="B1")
plt.plot(lambda1_values, B2_values, label="B2")
plt.xlabel("Интенсивность  $\lambda_1$ ")
plt.ylabel("Вероятность блокировки")
plt.title("Зависимость вероятности блокировки от  $\lambda_1$ ")
plt.grid()
plt.legend()
plt.show()

✓ [5] 213ms
```

Рисунок 3.5: Цикл расчета E, B1, B2 и график вероятностей блокировки

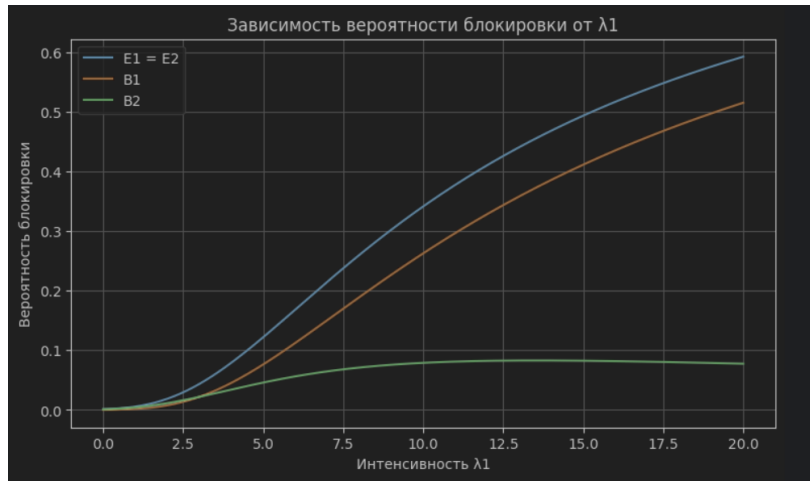


Рисунок 3.6: График вероятностей блокировки

3.6 Зависимость среднего числа запросов от интенсивности

Для второго графика используются те же значения λ_1 . На каждой итерации требуется только среднее число запросов N .

В строке

```
_, _, _, _, n_mean = erlang_model(C, l1, lambda2, mu)
```

из пяти возвращаемых функцией значений используется только последнее — среднее число запросов N . Остальные значения в этой части программы не нужны.

```

N_values = []

for l1 in lambda1_values:
    _, _, _, n_mean = erlang_model(C, l1, lambda2, mu)
    N_values.append(n_mean)

plt.figure(figsize=(9, 5))
plt.plot(lambda1_values, N_values)
plt.xlabel("Интенсивность  $\lambda_1$ ")
plt.ylabel("Среднее число запросов")
plt.title("Зависимость среднего числа запросов от  $\lambda_1$ ")
plt.grid()
plt.show()
✓ [6] 141ms

```

Рисунок 3.7: Цикл расчета N

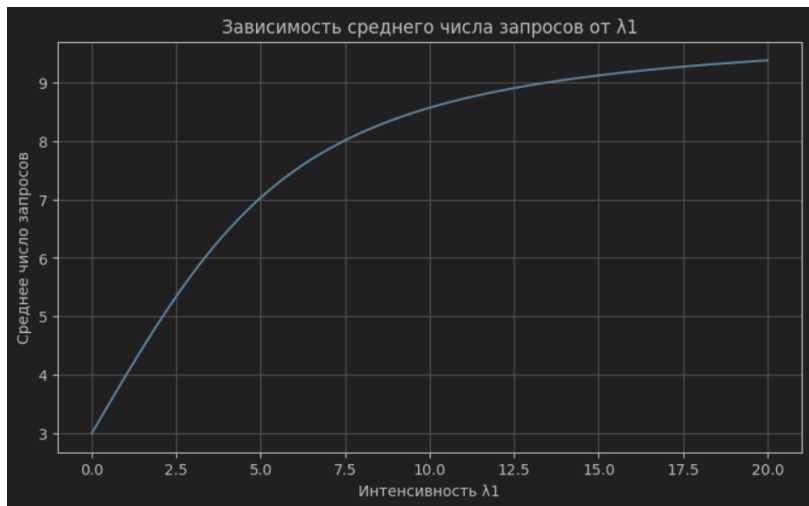


Рисунок 3.8: График среднего числа запросов

4 Вывод

В ходе лабораторной работы была реализована программа для расчета характеристик полнодоступной двухсервисной модели Эрланга.

Для заданных C , λ_1 , λ_2 и μ программа рассчитывает стационарное распределение вероятностей, вероятность блокировки по времени, вероятности блокировки по вызовам для двух типов запросов и среднее число запросов в системе.

При увеличении λ_1 общая нагрузка на систему увеличивается. В результате возрастает вероятность того, что все C каналов будут заняты, поэтому вероятность блокировки увеличивается. Среднее число одновременно обслуживаемых запросов также увеличивается и при большой нагрузке приближается к емкости системы C .